Выполнил: Василенко И.А Группа: ПИа-090303-о24

ЛАБОРАТОРНАЯ РАБОТА №4

Дисциплина: Основы программирования на Python

Тема: Модули, исключения, файлы и сериализация данных в Python

Вариант 2: Сотрудники

Цель работы

Получить практические навыки разработки многофайловых Python-приложений, обработки исключительных ситуаций и сохранения данных во внешних файлах. В процессе выполнения работы необходимо освоить модули и пакеты Python, импортирование классов и функций, организацию проекта по нескольким файлам, обработку исключений с помощью try, except, else и finally, создание собственных исключений, чтение и запись текстовых файлов, работу с форматами CSV и JSON, а также сериализацию и десериализацию объектов для сохранения состояния программы между запусками.

Постановка индивидуального задания (Вариант 2. Сотрудники)

Создать систему учёта сотрудников. Для каждого сотрудника хранить ФИО, должность, отдел и зарплату. Реализовать добавление сотрудника, поиск, изменение зарплаты, удаление, статистику зарплат и сохранение данных в JSON. Предусмотреть собственное исключение для попытки установить отрицательную зарплату. Программа должна использовать несколько модулей, содержать классы предметной области, обрабатывать некорректный пользовательский ввод, сохранять данные во внешний файл, загружать их при запуске, преобразовывать объекты в JSON и восстанавливать их обратно, а также предоставлять консольное меню.

Краткие теоретические сведения

Модулем в Python является любой файл с расширением .py. Модуль можно подключить целиком командой import models, обратившись затем к его содержимому через имя модуля, либо импортировать отдельные объекты командой from models import staff. Пакет представляет собой каталог с файлом __init__.py, который объединяет несколько модулей по функциональному признаку. Разделение программы на модули уменьшает размер отдельных файлов, позволяет повторно использовать код и разделять ответственность между частями приложения.

Исключением называется объект, который создаётся интерпретатором при возникновении ошибки во время выполнения программы. Конструкция try/except позволяет перехватить исключение и продолжить работу, блок else выполняется только при отсутствии ошибки, а блок finally — в любом случае. Оператор raise позволяет программе самостоятельно сформировать исключение, а наследование от класса Exception — объявить собственный тип ошибки, описывающий конкретную предметную ситуацию.

Для долговременного хранения данных используются внешние файлы. Текстовые файлы открываются функцией open() в режимах r, w, a и x; предпочтительным способом работы является контекстный менеджер with, автоматически закрывающий файл. Формат JSON хранит данные в виде вложенных словарей и списков и поддерживается модулем json, а формат CSV представляет данные в виде таблицы и поддерживается модулем csv. Сериализация — это преобразование объекта в словарь и далее в текст, десериализация — обратное восстановление объекта.

Структура проекта и назначение модулей

Программа разделена на пять модулей, каждый из которых отвечает за отдельную часть работы. Точкой входа является файл main.py, запускаемый командой python main.py.

Файл	Назначение и ключевые элементы
exceptions.py	Собственное исключение salaryError, наследующее класс Exception; используется при попытке задать отрицательную зарплату
models.py	Класс предметной области staff с полями fullname, position, department, salary; методы to_dict(), from_dict() и __str__()
services.py	Бизнес-логика: функции add_staff(), remove_staff(), search_staff(), calculate_salary_stats()
storage.py	Сохранение и загрузка данных: save_to_json(), load_from_json(), export_to_csv(); имена файлов employees.json и employees.csv
main.py	Консольное меню, ввод данных пользователем и обработка исключений ValueError и salaryError

Описание классов и функций

Класс staff описывает одного сотрудника. В конструкторе выполняется проверка корректности зарплаты: если переданное значение меньше нуля, возбуждается исключение salaryError с сообщением «Зарплата не может быть отрицательной». Значение зарплаты приводится к типу float, что позволяет хранить дробные суммы. Метод to_dict() преобразует объект в словарь с ключами fullname, position, department и salary, а метод from_dict(), объявленный как classmethod, создаёт новый экземпляр класса из такого словаря. Метод __str__ формирует строку для вывода сотрудника в консоль.

Модуль services.py содержит функции обработки коллекции сотрудников. Функция add_staff() создаёт объект staff и добавляет его в переданный список, функция remove_staff() удаляет сотрудника по ФИО без учёта регистра, функция search_staff() выполняет поиск по вхождению подстроки в ФИО или в название отдела, функция change_salary() изменяет зарплату сотрудника, найденного по ФИО, и проверяет, что новое значение неотрицательно, а функция calculate_salary_stats() рассчитывает суммарный фонд оплаты труда, среднюю, максимальную и минимальную зарплату и возвращает результат в виде словаря.

Модуль storage.py отвечает за обмен данными с файлами. Функция save_to_json() превращает каждый объект в словарь и записывает весь список в файл employees.json с параметрами ensure_ascii=False и indent=4, благодаря чему кириллица сохраняется в читаемом виде. Функция load_from_json() читает файл

и восстанавливает объекты через staff.from_dict(); если файла нет или он повреждён, перехватываются исключения FileNotFoundError и json.JSONDecodeError и возвращается пустой список, поэтому программа не завершается аварийно при первом запуске. Функция export_to_csv() выгружает данные в файл employees.csv с заголовком «ФИО, Должность, Отдел, Зарплата».

Модуль main.py реализует консольное меню из восьми пунктов: просмотр списка, добавление сотрудника, удаление, поиск, статистика зарплат, изменение зарплаты, экспорт в CSV и выход. При старте программы данные автоматически загружаются из JSON-файла и выводится количество загруженных записей. После добавления, удаления и изменения зарплаты состояние сразу сохраняется в файл, поэтому данные не теряются при закрытии программы.

Обработка исключительных ситуаций

В программе используются два уровня обработки ошибок. На уровне предметной логики отрицательная зарплата приводит к возбуждению собственного исключения salaryError в конструкторе класса staff. На уровне интерфейса ввод пользователя обёрнут в конструкцию try/except: исключение ValueError возникает при вводе нечислового значения зарплаты, а salaryError — при вводе отрицательного числа. Оба случая обрабатываются разными блоками except и сопровождаются понятными сообщениями, после чего программа возвращается в меню и продолжает работу.

Листинг 4.1 — Собственное исключение и проверка зарплаты (exceptions.py, models.py)

```
class salaryError(Exception):
    pass


class staff:
    def __init__(self, fullname, position, department, salary):
        if float(salary) < 0:
            raise salaryError("Зарплата не может быть отрицательной")
        self.fullname = fullname
        self.position = position
        self.department = department
        self.salary = float(salary)
```

Сериализация и сохранение состояния

Сохранение состояния построено по схеме «объект — словарь — JSON — словарь — объект». При записи каждый объект staff преобразуется методом to_dict() в словарь, список словарей сериализуется функцией json.dump(). При чтении функция json.load() возвращает список словарей, каждый из которых передаётся в classmethod from_dict(), и на выходе получается список полноценных объектов с работающими методами. Дополнительно реализован экспорт в CSV, который удобен для открытия данных в табличных редакторах.

Листинг 4.2 — Сохранение и загрузка данных (storage.py)

```
def save_to_json(employees_list):
    staff_list = []
    for emp in employees_list:
        staff_list.append(emp.to_dict())
    with open(JSON_FILE, "w", encoding="utf-8") as f:
        json.dump(staff_list, f, ensure_ascii=False, indent=4)


def load_from_json():
    try:
        with open(JSON_FILE, "r", encoding="utf-8") as f:
            dicts_list = json.load(f)
            employees_list = []
            for d in dicts_list:
                employees_list.append(staff.from_dict(d))
            return employees_list
    except (FileNotFoundError, json.JSONDecodeError):
        return []
```

Раздел тестирования

Проверка работоспособности выполнялась вручную по сценариям, перечисленным в методических указаниях. Сначала программа запускалась при отсутствии файла employees.json: загружалось ноль записей, что подтверждает корректную обработку исключения FileNotFoundError. Затем в меню добавлялись три сотрудника с разными должностями и отделами, после чего выполнялся поиск по названию отдела и выводилась статистика зарплат. Отдельно проверялось изменение зарплаты: корректное значение сохранялось в JSON и учитывалось в статистике, а попытка ввести отрицательную зарплату приводила к сообщению об ошибке и не изменяла данные.

№	Проверяемое действие	Ожидаемый и полученный результат
1	Первый запуск без файла данных	Загружено 0 сотрудников, программа работает
2	Добавление сотрудника с корректными данными	Запись добавлена в список и сохранена в employees.json
3	Ввод зарплаты текстом	Перехвачено исключение ValueError, выведено сообщение об ошибке
4	Ввод отрицательной зарплаты	Возбуждено salaryError, запись не создана
5	Поиск по отделу	Выведены все сотрудники указанного отдела
6	Удаление сотрудника по ФИО	Запись удалена, файл JSON обновлён
7	Расчёт статистики	Выведены фонд оплаты труда, средняя, максимальная и минимальная зарплата
8	Экспорт в CSV	Создан файл employees.csv с заголовком и четырьмя столбцами
9	Повторный запуск программы	Данные загружены из JSON автоматически

Отдельно проверялось содержимое файла employees.json: данные хранятся в виде массива объектов с четырьмя ключами, кириллица не экранируется, отступы обеспечивают читаемость. Повторный запуск программы подтвердил, что состояние сохраняется между сеансами работы.

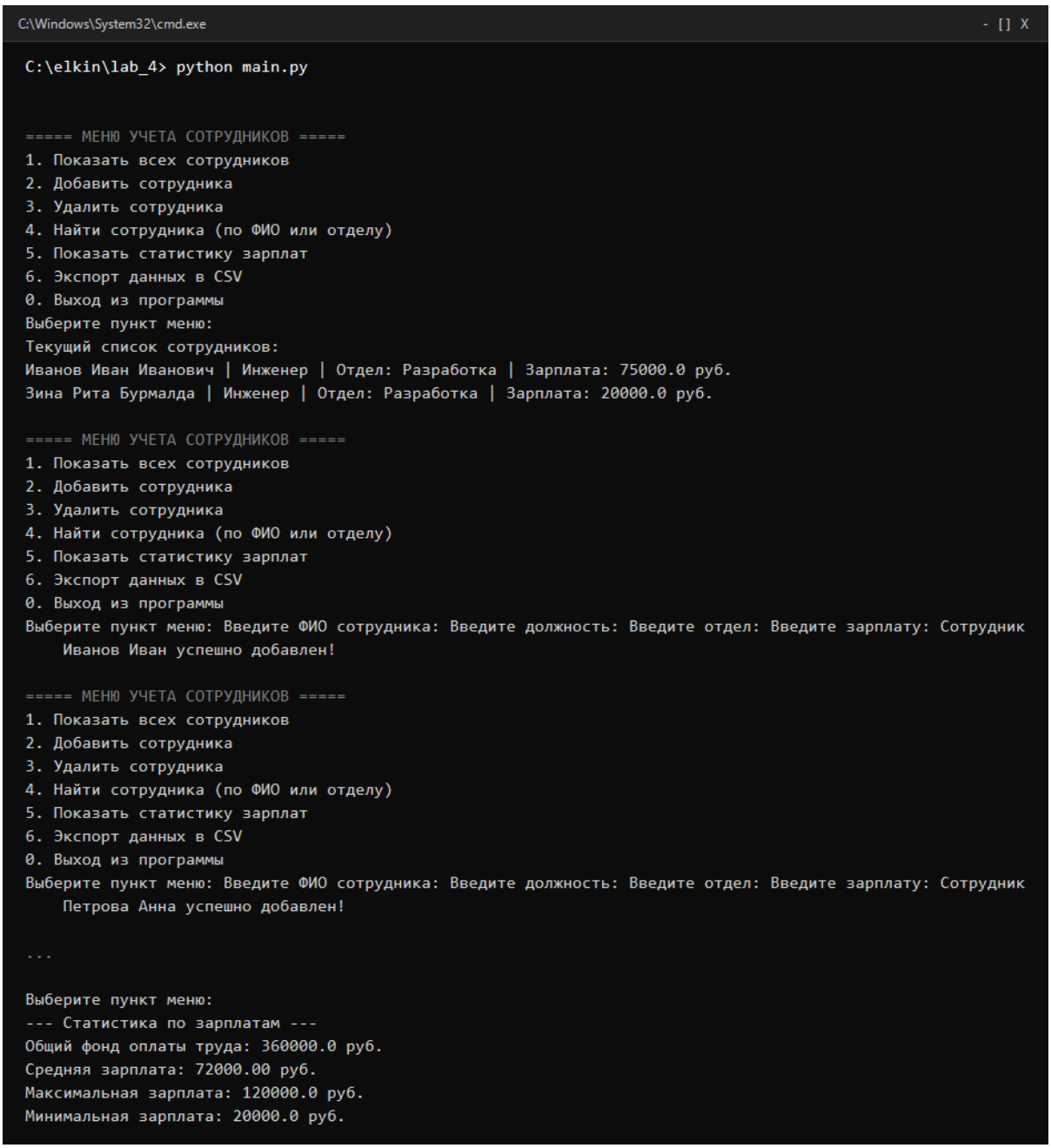


Рисунок 4.1 — Работа программы учёта сотрудников: меню, добавление сотрудника и статистика зарплат

Листинги ключевых фрагментов программы

Листинг 4.3 — Преобразование объекта в словарь и обратно (models.py)

```
def to_dict(self):
    return {
        "fullname": self.fullname,
        "position": self.position,
        "department": self.department,
        "salary": self.salary
    }

@classmethod
def from_dict(cls, data):
```

```
        return cls(
            fullname=data["fullname"],
            position=data["position"],
            department=data["department"],
            salary=data["salary"]
        )
```

Листинг 4.4 — Обработка ошибок ввода в консольном меню (main.py)

```
salary_input = input("Введите зарплату: ").strip()


try:
    services.add_staff(employees, fullname, position, department, salary_input)
    storage.save_to_json(employees)
except ValueError:
    print("Ошибка ввода: Зарплата должна быть числом!")
except salaryError as e:
    print(f"Ошибка бизнес-логики: {e}!")
```

Листинг 4.5 — Расчёт статистики зарплат (services.py)

```
def calculate_salary_stats(employees_list):
    if not employees_list:
        return None
    salaries = []
    for emp in employees_list:
        salaries.append(emp.salary)
    total = sum(salaries)
    avg = total / len(salaries)
    maximum = max(salaries)
    minimum = min(salaries)
    return {"total": total, "average": avg, "max": maximum, "min": minimum}
```

Вывод

В ходе выполнения лабораторной работы разработано многофайловое консольное приложение для учёта сотрудников. Программа разделена на модули моделей, бизнес-логики, хранения и представления, что позволило изолировать работу с файлами от предметных операций. Освоены механизмы импортирования, создания собственных исключений и их обработки в интерфейсе, а также сериализация объектов в JSON и их восстановление при запуске. Реализованный механизм сохранения состояния обеспечивает сохранность данных между сеансами работы, а дополнительный экспорт в CSV упрощает дальнейший анализ сведений о сотрудниках.